1 Méthodes de défense et attaques dans ByzFL

1.1 Agrégateurs

1.1.1 Average

Définition Elle est calculée avec la formule suivante :

$$\mathrm{Avg}(x_1, \ldots, x_n) = \frac{1}{n} \sum_{i=1}^{n} x_i$$

où $x_i \in \mathbb{R}^d$ représente le vecteur de gradient d'un client (honnête ou byzantin). Le vecteur final est obtenu en appliquant cette moyenne sur chacune des coordonnées des vecteurs reçus.

C'est un agrégateur classique, basé sur une opération linéaire, mais il est sensible aux attaques byzantines, notamment celles induisant des perturbations importantes.

1.1.2 Median

Définition L'agrégateur Median est défini par :

$$\mathrm{Median}(x_1, \ldots, x_n) = \left(\mathrm{med}(x_1^{(1)}, \ldots, x_n^{(1)}), \ldots, \mathrm{med}(x_1^{(d)}, \ldots, x_n^{(d)})\right)$$

où $x_i \in \mathbb{R}^d$ représente le vecteur de gradient d'un client (honnête ou byzantin) et med désigne la médiane. Le vecteur final est obtenu en calculant la médiane de chaque coordonnée parmi tous les vecteurs reçus.

Cet agrégateur statistique robuste est moins sensible aux valeurs aberrantes que la moyenne. Son intuition est qu'une valeur extrême influence peu la médiane tant qu'une majorité des clients restent honnêtes.

1.1.3 Trimmed Mean

Définition L'agrégateur Trimmed Mean (TrMean) est défini coordonnée par coordonnée. Pour chaque coordonnée, les valeurs sont triées, puis les f plus petites et les f plus grandes valeurs sont supprimées avant de calculer la moyenne :

$$\mathrm{TrMean}(x_1, \ldots, x_n)^{(k)} = \frac{1}{n - 2f} \sum_{i=f+1}^{n-f} x_{(i)}^{(k)}$$

où $x_{(i)}^{(k)}$ désigne la i-ème valeur de la coordonnée k après tri croissant, et f est le nombre maximal de clients byzantins supposés.

Le vecteur final est obtenu en appliquant cette opération à chaque coordonnée.

Cet agrégateur statistique robuste cherche à limiter l'influence des valeurs extrêmes en éliminant les plus petites et les plus grandes contributions avant le calcul de la moyenne. Il est une bonne solution face aux gradients byzantins se manifestant par des valeurs anormalement éloignées de celles des clients honnêtes.

1.1.4 Geometric Median

Définition L'agrégateur Geometric Median est défini comme la solution du problème d'optimisation suivant :

$$y^\star = \arg\min_{y \in \mathbb{R}^d} \sum_{i=1}^{n} \|y - x_i\|_2$$

où $x_1, \ldots, x_n \in \mathbb{R}^d$ représentent les gradients envoyés par les clients (honnêtes ou byzantins), et $\|\cdot\|_2$ désigne la norme euclidienne.

Le vecteur recherché $y^\star$ est donc le point de $\mathbb{R}^d$ qui minimise la somme des distances à tous les gradients. Il représente le centre le plus robuste du nuage de points formé par les vecteurs des apprenants.

Dans ByzFL, cette solution n'est pas calculée exactement. Elle est approchée numériquement à l'aide de l'algorithme de Weiszfeld lissé, qui transforme ce problème d'optimisation en une suite d'itérations.

Algorithme de Weiszfeld

$$y^{(t+1)} = \frac{\sum_{i=1}^{n} \frac{x_i}{\max(\|x_i - y^{(t)}\|_2, \nu)}}{\sum_{i=1}^{n} \frac{1}{\max(\|x_i - y^{(t)}\|_2, \nu)}}$$

Chaque itération consiste à calculer des poids inversement proportionnels aux distances entre $y^{(t)}$ et les points x_i, puis à effectuer une moyenne pondérée des x_i normalisée par la somme des poids, afin d'obtenir une nouvelle estimation $y^{(t+1)}$ plus proche du centre robuste des données.

Le paramètre ν sert à stabiliser les calculs lorsque des distances deviennent très petites (0.1 par défaut), et T correspond au nombre total d'itérations effectuées (3 par défaut). Après T étapes, on obtient une approximation de $y^\star$.

Intuition On peut imaginer les gradients comme un ensemble de points dans un espace. Le but est de trouver un point y qui se place au centre de ce nuage, non pas en faisant une simple moyenne, mais en minimisant les distances réelles à tous les points.

L'algorithme de Weiszfeld commence avec un point initial, puis le déplace progressivement. À chaque étape, les points proches du centre actuel ont plus d'influence que les points éloignés. Le point y glisse ainsi vers la zone la plus dense du nuage.

Contrairement à la moyenne, quelques points très éloignés n'ont pas une influence dominante. Le centre final reste donc proche du groupe majoritaire des gradients honnêtes.

1.1.5 Krum

Définition L'agrégateur Krum sélectionne directement un des vecteurs reçus comme résultat final :

$$\mathrm{Krum}(x_1, \ldots, x_n) = x_\lambda$$

où

$$\lambda \in \arg\min_{i \in [n]} \sum_{x_j \in N_i} \|x_i - x_j\|_2^2$$

et N_i représente l'ensemble des $n - f$ plus proches voisins de x_i parmi tous les vecteurs reçus.

Ici, $x_1, \ldots, x_n \in \mathbb{R}^d$ sont les gradients des clients (honnêtes ou byzantins), $\|\cdot\|_2$ désigne la norme euclidienne, et f représente le nombre maximal supposé de vecteurs byzantins.

Intuition Krum choisit le vecteur qui est le plus « cohérent » avec les autres. Pour chaque vecteur x_i, on regarde ses voisins les plus proches et on calcule à quel point il est éloigné d'eux. Le vecteur retenu est celui qui est globalement le plus proche d'un grand nombre d'autres gradients.

Interprétation Contrairement à la moyenne ou à la médiane géométrique, Krum ne construit pas un nouveau vecteur : il sélectionne directement un gradient existant, celui considéré comme le plus fiable selon les distances aux autres points.

1.1.6 Multi-Krum

Définition L'agrégateur Multi-Krum est une extension de Krum qui, au lieu de retenir un seul vecteur, sélectionne les $n - f$ vecteurs les plus fiables puis en calcule la moyenne :

$$\mathrm{MultiKrum}_f(x_1, \ldots, x_n) = \frac{1}{n-f} \sum_{i=1}^{n-f} x_{\pi(i)}$$

où π est une permutation sur $[n]$ qui trie les vecteurs d'entrée par ordre croissant de leur score de distance à leurs $n - f$ plus proches voisins :

$$\sum_{x \in N_{\pi(1)}} \|x_{\pi(1)} - x\|_2^2 \leq \cdots \leq \sum_{x \in N_{\pi(n)}} \|x_{\pi(n)} - x\|_2^2$$

et N_i désigne l'ensemble des $n - f$ plus proches voisins de x_i parmi $\{x_1, \ldots, x_n\}$.

Intuition Là où Krum ne garde qu'un seul vecteur (le meilleur), Multi-Krum garde les $n - f$ vecteurs les plus fiables et fait leur moyenne. Cela permet de conserver plus d'information utile tout en écartant les gradients les plus suspects.

1.1.7 Centered Clipping

Définition L'agrégateur Centered Clipping est défini de manière itérative. Le résultat final est v_L, calculé selon la récurrence suivante :

$$\text{CenteredClipping}_{m,L,\tau}(x_1, \ldots, x_n) = v_L$$

avec $v_0 = m$ et, pour tout $\ell \in \{0, \ldots, L-1\}$:

$$v_{\ell+1} = v_\ell + \frac{1}{n} \sum_{i=1}^{n} (x_i - v_\ell) \min\left(1, \frac{\tau}{\|x_i - v_\ell\|_2}\right)$$

où $x_1, \ldots, x_n \in \mathbb{R}^d$ sont les gradients des clients, $m \in \mathbb{R}^d$ est le point de départ de l'itération (vecteur nul par défaut), L est le nombre d'itérations (1 par défaut), $\tau > 0$ est le seuil de clipping (100.0 par défaut), et $\|\cdot\|_2$ désigne la norme euclidienne.

Lorsque l'instance est appelée plusieurs fois (typiquement à chaque tour d'entraînement), la valeur de m utilisée au tour suivant est égale au vecteur de sortie du tour précédent, ce qui permet d'ancrer l'agrégation sur l'historique de l'entraînement.

Intuition L'idée est de partir d'une estimation de référence v_0 et de la mettre à jour en intégrant les contributions des clients. Chaque gradient x_i est d'abord centré autour de v_ℓ (on calcule $x_i - v_\ell$), puis sa contribution est plafonnée, si x_i est trop éloigné de v_ℓ, son influence est réduite de sorte que sa contribution ait une norme au plus égale à τ. On fait ensuite la moyenne de toutes ces contributions rognées et on l'ajoute à v_ℓ.

Ainsi, les gradients byzantins très éloignés du centre courant ne peuvent pas faire dériver l'estimation de façon incontrôlée. L'agrégateur converge progressivement vers la région du nuage où se concentrent les gradients honnêtes.

1.1.8 Minimum Diameter Averaging (MDA)

Définition L'agrégateur MDA sélectionne le sous-ensemble de $n - f$ vecteurs dont le diamètre maximal est le plus petit, puis calcule leur moyenne :

$$\mathrm{MDA}_f(x_1, \ldots, x_n) = \frac{1}{n-f} \sum_{i \in S^\star} x_i$$

où

$$S^\star \in \arg \min_{\substack{S \subset \{1,\ldots,n\} \\ |S| = n-f}} \left\{ \max_{i,j \in S} \|x_i - x_j\|_2 \right\}$$

Ici, $x_1, \ldots, x_n \in \mathbb{R}^d$ sont les gradients des clients, f est le nombre supposé de vecteurs byzantins, et $\|\cdot\|_2$ désigne la norme euclidienne.

Intuition MDA cherche le groupe de $n-f$ vecteurs qui sont le plus « serrés » entre eux, c'est-à-dire le sous-ensemble dans lequel aucune paire de vecteurs n'est trop éloignée l'une de l'autre. L'hypothèse est que les gradients honnêtes forment naturellement un groupe cohérent, tandis que les vecteurs byzantins sont souvent éloignés de ce groupe ou les uns des autres. En retenant le sous-ensemble le plus compact, on maximise la probabilité d'exclure les gradients malveillants.

1.1.9 MoNNA

Définition L'agrégateur MoNNA calcule la moyenne des $n-f$ plus proches voisins d'un vecteur de référence choisi par son identifiant idx :

$$\mathrm{MoNNA}_{f,\mathrm{idx}}(x_1, \ldots, x_n) = \frac{1}{n-f} \sum_{i \in N_{\mathrm{idx}+1}} x_i$$

où N_i désigne l'ensemble des $n-f$ plus proches voisins de x_i dans $\{x_1, \ldots, x_n\}$, et $\mathrm{idx} \in \{0, \ldots, n-1\}$ est l'identifiant du vecteur de référence (0 par défaut).

MoNNA est principalement utilisé dans des contextes pair-à-pair (peer-to-peer), où idx correspond à l'identifiant d'un client de confiance, autrement dit, on sait que certains clients sont fiables, contrairement à d'autres agregateurs (krum, median) ou l'on sait pas qui est qui.

Intuition Plutôt que de chercher le vecteur le plus central parmi tous les clients, MoNNA part d'un client de confiance identifié et calcule la moyenne de ses $n-f$ voisins les plus proches. L'idée est que les voisins d'un client honnête sont eux-mêmes plus susceptibles d'être honnêtes, et que les clients byzantins, souvent éloignés du groupe honnête, seront naturellement exclus de ce voisinage.

1.1.10 Meamed

Définition L'agrégateur Meamed (Mean Around Median) est défini coordonnée par coordonnée. Pour chaque coordonnée k, on calcule la médiane de toutes les valeurs reçues, puis on retient les $n - f$ valeurs les plus proches de cette médiane pour en calculer la moyenne :

$$[\text{Meamed}_f(x_1, \ldots, x_n)]_k = \frac{1}{n-f} \sum_{j=1}^{n-f} \left[x_{\pi(j)}\right]_k$$

où π est une permutation qui trie les vecteurs selon leur distance croissante à la médiane de la coordonnée k :

$$\left|\left[x_{\pi(1)}\right]_k - \text{med}([x_1]_k, \ldots, [x_n]_k)\right| \leq \ldots \leq \left|\left[x_{\pi(n)}\right]_k - \text{med}([x_1]_k, \ldots, [x_n]_k)\right|$$

et $[\cdot]_k$ désigne la k-ième coordonnée.

Intuition Meamed combine les avantages de la médiane (robustesse aux valeurs extrêmes) et de la moyenne (précision). Pour chaque coordonnée, au lieu de simplement prendre la médiane comme résultat final, on identifie les $n - f$ valeurs les plus proches de la médiane et on en fait la moyenne. Cela permet d'obtenir un estimateur plus précis que la seule médiane, tout en restant robuste car les valeurs aberrantes byzantines, qui sont les plus éloignées de la médiane, sont exclues du calcul.

1.1.11 Covariance-bound Agnostic Filter (CAF)

Définition L'agrégateur CAF est un filtre robuste qui estime itérativement une moyenne en réduisant progressivement le poids des vecteurs qui s'écartent dans la direction la plus influente de la matrice de covariance empirique.

L'algorithme fonctionne de la manière suivante. On initialise des poids $c_i = 1$ pour tout $i \in [n]$. Ensuite, on répète les étapes suivantes jusqu'à ce que la somme des poids $\sum_i c_i \leq n - 2f$:

1. Calculer la moyenne empirique pondérée :

$$\mu_c = \frac{1}{\sum_i c_i} \sum_{i=1}^{n} c_i x_i$$

2. Calculer le vecteur propre dominant v et la valeur propre maximale $\lambda_{\max}$ de la matrice de covariance empirique pondérée (à l'aide de la méthode des puissances) :

$$\Sigma_c = \frac{1}{\sum_i c_i} \sum_{i=1}^{n} c_i (x_i - \mu_c)(x_i - \mu_c)^\top$$

3. Pour chaque vecteur, calculer la projection au carré sur v :

$$\tau_i = \left((x_i - \mu_c)^\top v\right)^2$$

4. Réduire le poids des vecteurs aberrants :

$$c_i \leftarrow c_i \cdot \left(1 - \frac{\tau_i}{\max_j \tau_j}\right)$$

À la fin, on retourne la moyenne pondérée μ_c correspondant à la plus petite valeur propre maximale $\lambda_{\max}$ observée au cours des itérations.

Intuition La matrice de covariance des gradients résume comment les gradients se dispersent dans l'espace. Sa valeur propre maximale et le vecteur propre associé indiquent la direction dans laquelle la dispersion est la plus grande. Or, les gradients byzantins ont tendance à être alignés dans une direction particulière pour maximiser leur perturbation.

À chaque étape, l'algorithme identifie cette direction de dispersion maximale et réduit le poids des vecteurs qui y contribuent le plus. Ainsi, les vecteurs byzantins, qui sont souvent les plus « saillants » dans cette direction, voient leur influence progressivement diminuer jusqu'à être quasiment ignorés. Le résultat final est la moyenne pondérée qui présente la plus faible dispersion.

1.1.12 Smallest Maximum Eigenvalue Averaging (SMEA)

Définition L'agrégateur SMEA sélectionne le sous-ensemble de $n - f$ vecteurs dont la matrice de covariance a la plus petite valeur propre maximale, puis retourne leur moyenne.

L'algorithme procède comme suit :

1. Énumérer tous les sous-ensembles $S \subset [n]$ de taille $n - f$.
2. Pour chaque sous-ensemble S, calculer sa moyenne empirique :

$$\mu_S = \frac{1}{|S|} \sum_{i \in S} x_i$$

3. Calculer la matrice de covariance empirique :

$$\Sigma_S = \frac{1}{|S|} \sum_{i \in S} (x_i - \mu_S)(x_i - \mu_S)^\top$$

4. Calculer la valeur propre maximale $\lambda_{\max}(\Sigma_S)$ de chaque sous-ensemble (à l'aide de la méthode des puissances).

5. Sélectionner le sous-ensemble $S^\star$ qui minimise cette valeur propre maximale :
$$S^\star = \arg \min_{S\,:\,|S|=n-f} \lambda_{\max}(\Sigma_S)$$

6. Retourner la moyenne empirique du sous-ensemble optimal :
$$\mathrm{SMEA}(x_1, \ldots, x_n) = \frac{1}{|S^\star|} \sum_{i \in S^\star} x_i$$

Cet agrégateur est coûteux en calcul en raison de son caractère combinatoire, mais il offre des garanties de robustesse de premier plan. Il est donc particulièrement adapté aux contextes fédérés avec un nombre de clients pas trop grand.

Intuition Un groupe de gradients honnêtes, calculés sur des données similaires, forment un nuage relativement compact : leur covariance a une faible valeur propre maximale. À l'inverse, un groupe contenant des gradients byzantins sera plus dispersé et aura une valeur propre maximale plus grande. SMEA cherche donc exhaustivement le groupe de $n-f$ gradients dont la dispersion est la plus faible, ce qui revient à trouver le groupe le plus homogène et donc le plus probablement constitué de clients honnêtes.

1.2 Pré-agrégateurs

Les pré-agrégateurs sont des transformations appliquées sur l'ensemble des vecteurs reçus *avant* de les passer à l'agrégateur principal. Leur rôle est de normaliser ou de restructurer les données afin de renforcer la robustesse de l'agrégation.

1.2.1 Nearest Neighbor Mixing (NNM)

Définition Le pré-agrégateur NNM remplace chaque vecteur x_i par la moyenne de ses $n - f$ plus proches voisins dans l'ensemble reçu :

$$\mathrm{NNM}_f(x_1, \ldots, x_n) = \left(\frac{1}{n-f} \sum_{i \in N_1} x_i \ , \ \ldots \ , \ \frac{1}{n-f} \sum_{i \in N_n} x_i \right)$$

où N_i est l'ensemble des $n-f$ plus proches voisins de x_i dans $\{x_1, \ldots, x_n\}$, et f est le nombre supposé de vecteurs byzantins (0 par défaut).

La sortie est donc un ensemble de n nouveaux vecteurs (un par client original), chacun étant la moyenne de son voisinage.

Intuition En remplaçant chaque gradient par la moyenne de ses voisins les plus proches, NNM « lisse » les données avant l'agrégation. Un gradient honnête, entouré d'autres gradients honnêtes proches, sera peu modifié. En revanche, un gradient byzantin, souvent isolé ou entouré d'autres vecteurs byzantins qui ne ressemblent pas aux gradients honnêtes, sera davantage influencé par ses voisins. L'idée est de réduire l'hétérogénéité artificielle introduite par les attaquants avant l'étape d'agrégation finale.

1.2.2 Bucketing

Définition Le pré-agrégateur Bucketing répartit aléatoirement les n vecteurs en groupes (buckets) de taille s, puis remplace chaque groupe par la moyenne de ses éléments :

$$\text{Bucketing}_s(x_1, \ldots, x_n) = \left(\frac{1}{s} \sum_{i=1}^{s} x_{\pi(i)} \ , \ \frac{1}{s} \sum_{i=s+1}^{2s} x_{\pi(i)} \ , \ \ldots \ , \ \frac{1}{s} \sum_{i=(\lceil n/s \rceil - 1)s+1}^{n} x_{\pi(i)} \right)$$

où π est une permutation aléatoire sur $[n]$, et $s > 0$ est la taille des buckets (1 par défaut). La sortie est donc un ensemble de $\lceil n/s \rceil$ vecteurs.

Intuition En regroupant aléatoirement les gradients et en calculant leur moyenne par groupe, Bucketing réduit l'hétérogénéité des données. Si chaque bucket contient en majorité des clients honnêtes (ce qui est probable quand f est petit devant n), la moyenne du bucket sera proche des gradients honnêtes. Cela rend plus difficile pour un attaquant de dominer un bucket, et améliore la robustesse des agrégateurs qui suivent, notamment dans des contextes de forte hétérogénéité des données.

1.2.3 Static Clipping

Définition Le pré-agrégateur Static Clipping applique à chaque vecteur un rognage basé sur un seuil fixe c : tout vecteur dont la norme dépasse c est redimensionné pour avoir une norme exactement égale à c :

$$\text{Clipping}_c(x_1, \ldots, x_n) = \left(\min\left\{1, \frac{c}{\|x_1\|_2}\right\} x_1 \ , \ \ldots \ , \ \min\left\{1, \frac{c}{\|x_n\|_2}\right\} x_n \right)$$

où $c \geq 0$ est le seuil de clipping statique (2.0 par défaut) et $\|\cdot\|_2$ désigne la norme euclidienne. Si $\|x_i\|_2 \leq c$, le vecteur est laissé intact ; sinon, il est multiplié par $c/\|x_i\|_2$ pour ramener sa norme à c.

Intuition Les attaques byzantines utilisent souvent des gradients de très grande norme pour dominer l'agrégation. En bornant la norme de chaque vecteur par un seuil fixe c, le Static Clipping limite l'influence maximale qu'un seul vecteur peut avoir sur le résultat final. Le seuil c est dit « statique » car il est fixé à l'avance et ne s'adapte pas aux données reçues.

1.2.4 Adaptive Robust Clipping (ARC)

Définition Le pré-agrégateur ARC est une version adaptative du clipping : le seuil est déterminé automatiquement à partir des données reçues, en fonction du nombre de clients byzantins supposés. Chaque vecteur est rogné par un seuil calculé à partir de la k-ième plus grande norme parmi les vecteurs reçus :

$$\mathrm{ARC}_f(x_1, \ldots, x_n) = \left(\min\left\{1, \frac{\|x_{\pi(k)}\|_2}{\|x_1\|_2}\right\} x_1 \ , \ \ldots \ , \ \min\left\{1, \frac{\|x_{\pi(k)}\|_2}{\|x_n\|_2}\right\} x_n \right)$$

où π est une permutation qui trie les vecteurs par norme croissante ($\|x_{\pi(1)}\|_2 \leq \ldots \leq \|x_{\pi(n)}\|_2$), et $k = \lfloor 2 \cdot \frac{f}{n} \cdot (n - f) \rfloor$.

Le seuil de clipping utilisé est donc $\|x_{\pi(k)}\|_2$, la k-ième plus petite norme parmi les gradients reçus.

Intuition Contrairement au Static Clipping qui utilise un seuil fixé à l'avance, ARC détermine le seuil à partir des données elles-mêmes. L'indice k est calibré de manière à ce que le seuil corresponde à une norme représentative des gradients les plus modestes, ce qui permet d'éliminer l'influence démesurée de vecteurs de grande norme pouvant être byzantins, tout en s'adaptant à la distribution réelle des gradients reçus à chaque tour d'entraînement.

2 Attaques

Les attaques byzantines sont des stratégies utilisées par des clients malveillants pour perturber le processus d'agrégation et dégrader les performances du modèle appris.

2.1 Sign Flipping

Définition L'attaque Sign Flipping envoie l'opposé de la moyenne des gradients honnêtes :

$$\mathrm{SignFlipping}(x_1, \ldots, x_n) = -\frac{1}{n} \sum_{i=1}^{n} x_i$$

où $x_1, \ldots, x_n$ sont les gradients corrects envoyés par les clients honnêtes lors d'une itération d'entraînement.

Cette attaque vise à faire converger le modèle dans la mauvaise direction en introduisant un gradient exactement opposé à la moyenne des gradients honnêtes.

2.2 Inner Product Manipulation (IPM)

Définition L'attaque IPM multiplie la moyenne des gradients honnêtes par $-\tau$:

$$\text{IPM}_\tau(x_1, \ldots, x_n) = -\tau \cdot \frac{1}{n} \sum_{i=1}^{n} x_i$$

où $x_1, \ldots, x_n$ sont les gradients corrects des clients honnêtes, et $\tau > 0$ est le facteur d'attaque (2.0 par défaut).

IPM est une généralisation de Sign Flipping : en faisant varier τ, l'attaquant peut régler l'amplitude de la perturbation. Cette attaque est conçue pour que le produit scalaire entre le gradient byzantin et la moyenne des gradients honnêtes soit négatif et de grande magnitude, orientant ainsi le modèle dans la mauvaise direction.

2.3 Optimal Inner Product Manipulation (Opt-IPM)

Définition L'attaque Opt-IPM est une généralisation de IPM qui optimise automatiquement le facteur τ pour maximiser la perturbation induite sur un agrégateur et un ensemble de pré-agrégateurs donnés :

$$\text{Opt-IPM}_{agg, pre_agg_list, f}(x_1, \ldots, x_n) = -\tau_{opt} \cdot \frac{1}{n} \sum_{i=1}^{n} x_i$$

où $\tau_{opt} \in \mathbb{R}$ est le facteur d'attaque optimal trouvé par une recherche linéaire (*line-search*). La fonction à maximiser est la norme ℓ_2 de la distance entre le vecteur agrégé (en présence des vecteurs byzantins) et la moyenne des gradients honnêtes.

Les paramètres de l'optimisation sont : le nombre d'évaluations maximales `evals` (20 par défaut), le facteur initial `start` (0.0 par défaut), le pas initial δ (10.0 par défaut) et le ratio de contraction `ratio` (0.8 par défaut).

Intuition Plutôt que de fixer τ à la main, Opt-IPM cherche automatiquement la valeur de τ qui cause le plus grand écart possible entre le gradient agrégé et ce que le serveur devrait obtenir en l'absence d'attaque. L'attaque s'adapte ainsi à la défense utilisée.

2.4 A Little Is Enough (ALIE)

Définition L'attaque ALIE perturbe la moyenne des gradients honnêtes en y ajoutant un bruit calibré sur l'écart-type coordonnée par coordonnée :

$$\text{ALIE}_\tau(x_1, \ldots, x_n) = \mu_{x_1,\ldots,x_n} + \tau \cdot \sigma_{x_1,\ldots,x_n}$$

où $\mu_{x_1,\ldots,x_n} = \frac{1}{n}\sum_{i=1}^{n} x_i$ est le vecteur moyenne, $\tau \in \mathbb{R}$ est le facteur d'attaque (1.5 par défaut), et $\sigma_{x_1,\ldots,x_n}$ est l'écart-type coordonnée par coordonnée, défini par :

$$[\sigma_{x_1,\ldots,x_n}]_k = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left([x_i]_k - [\mu_{x_1,\ldots,x_n}]_k\right)^2}$$

Intuition L'idée derrière ALIE est que le vecteur byzantin ne doit pas être trop éloigné des gradients honnêtes (pour ne pas être éliminé par des défenses basées sur la distance), mais doit tout de même être suffisamment perturbé pour nuire à l'entraînement. En perturbant chaque coordonnée proportionnellement à son écart-type, l'attaque produit un vecteur qui se situe à la limite des intervalles de confiance des agrégateurs robustes, d'où son nom : « un peu suffit ».

2.5 Optimal A Little Is Enough (Opt-ALIE)

Définition L'attaque Opt-ALIE est une généralisation de ALIE qui optimise automatiquement le facteur τ de la même manière qu'Opt-IPM optimise celui de IPM :

$$\text{Opt-ALIE}_{agg,pre_agg_list,f}(x_1, \ldots, x_n) = \mu_{x_1,\ldots,x_n} + \tau_{opt} \cdot \sigma_{x_1,\ldots,x_n}$$

où τ_{opt} est le facteur optimal trouvé par recherche linéaire en maximisant la norme ℓ_2 de la distance entre le vecteur agrégé et la moyenne des gradients honnêtes. Les paramètres de l'optimisation sont les mêmes que pour Opt-IPM.

2.6 Mimic

Définition L'attaque Mimic consiste pour le client byzantin à envoyer exactement le même vecteur qu'un client honnête désigné par son identifiant ϵ :

$$\text{Mimic}_\epsilon(x_1, \ldots, x_n) = x_{\epsilon+1}$$

où $x_1, \ldots, x_n$ sont les gradients corrects des clients honnêtes, et $\epsilon \in \{0, \ldots, n-1\}$ est l'identifiant du client imité (0 par défaut).

Cette attaque est utilisée principalement dans des contextes pair-à-pair pour évaluer la robustesse des défenses basées sur les identités des clients.

2.7 Infinity

Définition L'attaque Infinity envoie un vecteur dont toutes les coordonnées sont égales à $+\infty$:

$$\mathrm{Inf}(x_1, \ldots, x_n) = \begin{bmatrix} +\infty \\ +\infty \\ \vdots \\ +\infty \end{bmatrix} \in \mathbb{R}^d$$

où d est la dimension des vecteurs d'entrée. Cette attaque teste la robustesse des agrégateurs face à des valeurs numériquement extrêmes.

2.8 Gaussian

Définition L'attaque Gaussian génère un vecteur dont chaque coordonnée est tirée indépendamment d'une loi gaussienne de moyenne μ et d'écart-type σ :

$$\mathrm{Gaussian}_{\mu,\sigma}(x_1, \ldots, x_n) = \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_d \end{bmatrix} \in \mathbb{R}^d$$

où $g_i \sim \mathcal{N}(\mu, \sigma^2)$ pour tout $i \in \{1, \ldots, d\}$, avec $\mu = 0.0$ par défaut et $\sigma = 1.0$ par défaut. Cette attaque modélise un bruit aléatoire non structuré et sert de référence pour évaluer la robustesse des défenses face à des perturbations non coordonnées.